

Enterprise AI Architect Learning Path v3 – Phase Guide 5

Enterprise AI Architect Learning Path v3 – Phase Guide 5

MCP & Tool Integration

Purpose of This Phase

Phase 4 introduced agent orchestration, state management, and workflow design.

Phase 5 focuses on one of the most important developments in modern AI architecture:

Giving agents controlled access to external capabilities.

Large Language Models are valuable because they can reason.

Agents become valuable when they can act.

Modern AI agents rarely operate in isolation. Instead, they interact with:

- APIs
- Databases
- Search systems
- Business applications
- External services
- Enterprise platforms

This phase introduces the architectural patterns that allow those interactions to be secure, observable, reusable, and governable.

The primary focus is the **Model Context Protocol (MCP)** and enterprise tool architecture.

Why This Phase Matters

Without tools, agents are limited to:

- Generating text
- Summarizing information
- Answering questions

With tools, agents can:

- Query systems
- Retrieve information
- Execute workflows

- Update records
- Coordinate services

The challenge is balancing capability with control.

Enterprise architects must ensure that:

- Tools are secure
 - Access is governed
 - Actions are auditable
 - Permissions are enforced
 - Failures are predictable
-

Phase Objectives

Upon completion of this phase, you should be able to:

- Explain MCP architecture
 - Build MCP servers
 - Design tool contracts
 - Implement secure tool access
 - Define permission boundaries
 - Separate orchestration from execution
 - Create reusable tool ecosystems
 - Implement audit trails
 - Design agent-access governance models
 - Create enterprise-ready tool infrastructures
-

Architect Mindset

The most important principle of this phase is:

Tools are enterprise assets.

Avoid thinking:

"What tool does my agent need?"

Instead ask:

"What reusable capability should the organization expose?"

Enterprise architecture is usually about creating shared capabilities rather than solving a single workflow.

Primary Resources

Required

R12

Model Context Protocol Documentation

Focus Areas:

- Hosts
 - Clients
 - Servers
 - Tools
 - Resources
 - Prompts
-

R13

MCP Specification

Focus Areas:

- JSON-RPC
 - Tool Contracts
 - Protocol Definitions
 - Security Boundaries
-

R14

OWASP Top 10 for LLM Applications

Focus Areas:

- Tool misuse
 - Prompt injection
 - Excessive autonomy
 - Execution risks
-

R8

Semantic Kernel Documentation

Focus Areas:

- Tool wrappers
- Plugins
- Function calling

R10

LangGraph Documentation

Focus Areas:

- Tool integration
 - Workflow execution
 - Agent capabilities
-

Core Technical Skills

1. Tool Architecture

Understand the distinction between:

Agent

Makes decisions.

Tool

Performs actions.

Data Source

Stores information.

Workflow

Coordinates interactions.

Good architecture keeps these concerns separate.

2. MCP Fundamentals

Understand the major MCP roles.

Host

The system interacting with tools.

Examples:

- Agent frameworks
 - Desktop AI clients
 - Enterprise copilots
-

Client

Requests capabilities.

Server

Provides capabilities.

Tool

A specific callable function.

Resource

Information exposed to agents.

3. JSON-RPC

Learn:

- Requests
- Responses
- Parameters
- Error handling

You do not need to become a protocol expert.

You need to be capable of:

- Reading specifications
 - Building compliant tools
 - Troubleshooting integrations
-

4. Tool Contracts

Every tool should clearly define:

Inputs

Example:

```
{  
  "rule_name": "grappling"  
}
```

Outputs

Example:

```
{  
  "source": "rules.md",  
  "result": "..."  
}
```

Errors

Example:

```
{  
  "error": "rule_not_found"  
}
```

Poorly defined contracts become difficult to govern.

5. Permission Models

Enterprise systems need clear boundaries.

Examples:

Read Access

Allowed

Search Access

Allowed

Update Access

Restricted

Administrative Actions

Human approval required

Permission models should exist independently of the LLM.

6. Auditability

Every tool invocation should be traceable.

Capture:

- Tool name
- Request
- Response
- User
- Timestamp
- Outcome

Architects should assume audits will occur.

MCP vs Traditional APIs

A common misconception is:

MCP replaces APIs.

It does not.

A more accurate view:

API

↓

Capability

↓

MCP Tool

↓

Agent

MCP standardizes how agents discover and use capabilities.

The underlying systems often remain the same.

Phase Project

RPG MCP Server

The first enterprise-grade tool platform within the portfolio.

Its purpose is to expose RPG Catalog capabilities through a reusable protocol.

Functional Requirements

Rules Lookup

Provide:

search_rules

``

capability.

The Virtual GM should retrieve information through tools rather than hidden dependencies.

Lore Retrieval

Provide:

search_lore

capability.

Metadata Query

Provide:

search_sources

capability.

Tool Discovery

Clients should be able to understand:

- Available tools
- Inputs
- Outputs

without examining code.

Security Requirements

Least Privilege

Tools should only expose necessary functionality.

Validation

Validate:

- Inputs
- Parameters
- Types

before execution.

Execution Controls

Prevent:

- Arbitrary execution
 - Unauthorized actions
 - Prompt-based privilege escalation
-

Logging

All requests should be logged.

Architecture Deliverables

ADR-013

MCP Adoption Strategy

Document:

- Why MCP was selected
 - Benefits
 - Tradeoffs
-

ADR-014

Tool Contract Design

Document:

- Tool definitions
 - Input models
 - Output models
-

ADR-015

Permission Model

Document:

- Roles
 - Permissions
 - Governance rules
-

Portfolio Artifacts

MCP Architecture Diagram

Illustrate:

Virtual GM

↓

MCP Client

↓

MCP Server

↓

RPG Catalog

Tool Registry

Document:

- Tool Name
 - Purpose
 - Inputs
 - Outputs
-

Permission Matrix

Define:

- Access levels
 - Security boundaries
 - Approval requirements
-

Audit Strategy Document

Describe:

- Logging
- Monitoring
- Review procedures

Suggested Sprint Progression

Sprint 1 — MCP Fundamentals

Goals:

- Understand protocol
- Review architecture

Done When:

- MCP architecture understood
-

Sprint 2 — Server Foundation

Goals:

- Create MCP server
- Implement core structure

Done When:

- Server running
-

Sprint 3 — Rules Tool

Goals:

- Expose rule lookup

Done When:

- Tool functioning
-

Sprint 4 — Lore Tool

Goals:

- Expose lore search

Done When:

- Tool functioning
-

Sprint 5 — Security Controls

Goals:

- Validation
- Permissions
- Logging

Done When:

- Security model implemented
-

Sprint 6 — Agent Integration

Goals:

- Connect Virtual GM

Done When:

- Agent successfully consumes tools
-

Sprint 7 — Hardening

Goals:

- Testing
- Documentation
- Governance

Done When:

- RPG MCP Server v1 completed
-

Common Pitfalls

Pitfall 1

Giving Agents Direct Access to Everything

Enterprise systems require boundaries.

Not every capability should become a tool.

Pitfall 2

Ignoring Governance

Capabilities should have ownership.

Ask:

Who is responsible for this tool?

Pitfall 3

Coupling Tools to One Agent

Tools should be reusable.

Design for multiple consumers.

Pitfall 4

Skipping Audit Trails

Invisible actions are difficult to govern.

Every tool invocation should be observable.

Pitfall 5

Treating MCP as the Goal

MCP is a protocol.

The real goal is building a reusable enterprise capability layer.

Enterprise Translation Layer

The MCP architecture demonstrated in this project maps directly to enterprise environments.

	≡ RPG MCP Capability	≡ Enterprise Equivalent
1	Rules Lookup	Policy Lookup
2	Lore Search	Knowledge Search
3	Source Retrieval	Document Retrieval
4	Virtual GM	Operations Agent

5	RPG Catalog	Enterprise Knowledge Platform
6	Tool Registry	Enterprise Capability Registry

The implementation differs, but the architecture remains largely the same.

Definition of Done

Phase 5 is complete when:

- MCP architecture is understood
 - MCP server is operational
 - Tool contracts are documented
 - Permission model exists
 - Security controls are implemented
 - Audit strategy exists
 - Virtual GM successfully uses MCP tools
 - Architecture diagrams are completed
 - ADRs are documented
 - RPG MCP Server v1 is operational
-

End-of-Phase Self-Assessment

You are ready for Phase 6 when you can confidently answer:

- Can I explain MCP architecture?
- Can I design reusable tool ecosystems?
- Can I separate reasoning from execution?
- Can I create secure tool contracts?
- Can I design permission and governance models?
- Can I explain how enterprise agents should interact with external capabilities?

If all answers are "yes," proceed to **Phase 6 – AI Publishing Company Platform**.